

Checkout transactionnel complet

Order Saga + Tax + Fraud/Risk + Payment + Hyperswitch

Objectif

Donner a une IA de developpement un cahier des charges executable pour construire le premier flux financier critique de la plateforme e-commerce, sans ambiguite sur les responsabilites, la fiabilite, les compensations, la securite et les tests.

Regle majeure

Toute decision critique est backend-authoritative. Le frontend ne confirme jamais un paiement, une commande, une reservation de stock ou un remboursement sur la seule base d'un etat client.

Domaine	Technologies imposees
Backend	Go, REST/JSON, gRPC, Protocol Buffers/Buf
Data	PostgreSQL via pgx + sqlc, PgBouncer, CloudNativePG
Evenements	Kafka/Strimzi + Outbox Pattern + Apicurio Registry
Jobs	RabbitMQ pour traitements asynchrones operationnels si necessaire
Paiement	Payment Service + Hyperswitch + PSP final externe
Observabilite	OpenTelemetry, Prometheus, Loki, Tempo, Grafana
Delivery	Tekton, Harbor, Cosign, FluxCD, Flagger

Ce document doit etre suivi comme un plan d'implementation, pas comme une simple description.

1. Mission de l'IA

Construire un vertical slice de checkout reellement compilable, testable et deployable. Ne pas generer un prototype jetable. Le code doit etre propre, factorise, maintenable, observable et conforme aux decisions d'architecture deja fixees.

- Proceder par petites etapes, avec une compilation et des tests a chaque etape.
- Ne jamais dupliquer la logique metier entre REST et gRPC : les deux transports appellent les memes use cases.
- Ne pas introduire de nouvelle technologie sans justification et validation explicite.
- Tout acces reseau a un timeout ; tout retry est borne et reserve aux operations idempotentes.
- Aucune operation critique ne doit reposer sur Redis comme source de verite.
- Aucun appel externe ou gRPC ne doit etre effectuee a l'interieur d'une transaction PostgreSQL.
- Aucun secret longue duree dans Git ou Kubernetes ; utiliser OpenBao runtime.

Livrable attendu de l'IA

Un repository fonctionnel avec code, migrations, contrats, tests, Dockerfiles, manifests GitOps, pipelines CI et documentation minimale.
Chaque milestone doit finir par: build PASS + tests PASS + lint PASS.

2. Architecture fonctionnelle cible

```
Cart
-> Checkout API
-> Order Saga Orchestrator
-> Pricing
-> Tax
-> Inventory reservation
-> Fraud/Risk
-> Payment
-> Hyperswitch
-> PSP
-> Confirm Order

Events durables -> Kafka
Jobs operationnels -> RabbitMQ
```

L'Order Saga Orchestrator est responsable de l'enchaînement, des timeouts, du suivi d'etat et des compensations. Kafka transporte les faits metier durables, mais ne remplace pas l'orchestrateur pour le workflow financier.

3. Services a implementer ou completer

Service	Responsabilite
Order	Creation de commande, orchestration Saga, snapshot immutable, etats et compensations
Tax	Calcul fiscal versionne, arrondis, B2B/B2C si applicable, aucun ownership du prix commercial
Fraud/Risk	Scoring, regles antifraude, decision ALLOW/REVIEW/DENY
Payment	Autorisation/capture/refund, idempotence, references PSP, aucun PAN/CVV
Pricing	Recalcul final du prix effectif au checkout
Inventory	Reservation, commit/release du stock
Hyperswitch adapter	Orchestration de PSP via interface/adapter, aucune contamination du domaine

4. Structure Go obligatoire

```
services/order/  
cmd/  
internal/domain/  
internal/application/  
internal/infrastructure/  
internal/transport/  
api/  
openapi/  
proto/  
migrations/  
tests/
```

Meme structure pour tax, fraud-risk et payment.

Les librairies communes doivent rester minimales : observabilite, enveloppe evenementielle, utilitaires de transport, erreurs standardisees. Ne pas partager de logique metier entre domaines.

5. Modele Order et machine d'etat

Le modele doit etre une vraie machine d'etat. Une transition non autorisee doit etre rejetee par le domaine.

```
DRAFT  
-> PENDING  
-> STOCK_RESERVED  
-> RISK_APPROVED  
-> PAYMENT_AUTHORIZED  
-> CONFIRMED  
  
Branches d'echec:  
PENDING -> FAILED / CANCELLED  
STOCK_RESERVED -> PAYMENT_FAILED / CANCELLED  
PAYMENT_AUTHORIZED -> REFUNDED (selon workflow)
```

Ne jamais modifier un statut par simple UPDATE arbitraire. Exposer des commandes de domaine explicites : ReserveStockSucceeded, RiskApproved, PaymentAuthorized, Confirm, Cancel, Fail, etc.

6. Checkout API et idempotence HTTP

```
POST /api/v1/checkout  
Idempotency-Key: <uuidv7>
```

L'idempotence doit etre persistante en PostgreSQL avec contrainte UNIQUE. Une meme cle avec un payload different doit etre rejetee.

Champ idempotence	Usage
idempotency_key	cle fournie par le client
customer_id	contexte utilisateur
operation	checkout / authorize / capture / refund
request_hash	detecter une reutilisation avec un autre payload
status/result	rejouer le resultat precedent
created_at/expires_at	cycle de vie

Un double clic, un retry du navigateur, un timeout proxy ou un retry BFF ne doivent jamais creer deux commandes ou deux paiements.

7. Sequence transactionnelle du checkout

1. Recevoir CheckoutCommand + Idempotency-Key
2. Verifier/creer l'enregistrement d'idempotence
3. Charger Cart
4. Revalider chaque SKU
5. Pricing: recalculer le prix final
6. Tax: calculer les taxes

7. Creer Order = PENDING + snapshots
8. Inventory: reserver le stock
9. Fraud/Risk: obtenir ALLOW/REVIEW/DENY
10. Payment: autoriser/capturer selon strategie
11. Confirmer Order
12. Publier les evenements via Outbox/Kafka
13. Retourner l'etat backend confirme au frontend

Le Cart ne fait pas foi pour le prix final. Pricing et Tax recalculent au checkout. L'Order stocke un snapshot immutable du prix, des remises, des taxes et du total final.

8. Snapshots immuables

Snapshot	Contenu minimal
Pricing	prix unitaire, remises, promotions/coupons appliques, pricing_version
Tax	tax lines, taux, juridiction/regle, arrondis, tax_version
Order item	sku_id, quantity, unit_price, discount, tax, line_total
Order total	subtotal, discount_total, tax_total, grand_total, currency

Les changements futurs de prix, promotions ou regles fiscales ne modifient jamais une commande existante.

9. Inventory reservation

```
ReserveInventory(order_id, items, idempotency_key)
-> RESERVED
-> OUT_OF_STOCK

Compensation:
ReleaseInventory(order_id)
```

La reservation doit etre transactionnelle, durable et idempotente. Redis peut acclerer des lectures mais ne doit jamais garantir la reservation critique.

10. Fraud/Risk

Fraud/Risk retourne une decision explicite et versionnee : ALLOW, REVIEW ou DENY. La Saga doit traiter chaque cas.

```
ALLOW -> continuer vers Payment
REVIEW -> etat de revue / politique explicite
DENY -> ne pas appeler Payment, liberer le stock, Order FAILED/CANCELLED
```

Le mode fail-open/fail-closed en cas d'indisponibilite de Fraud/Risk doit etre configure par politique, pas improvise dans le code.

11. Payment + Hyperswitch

Payment est le proprietaire des transactions. Hyperswitch est une couche d'orchestration technique des PSP. Le domaine Payment ne doit pas dependre des objets propres a un fournisseur.

```
Payment Domain
-> PaymentProvider Port
-> HyperswitchAdapter
-> Hyperswitch
-> PSP A / PSP B
```

Payment conserve	Payment ne conserve jamais
payment_id	PAN complet
psp_reference	CVV

Payment conserve	Payment ne conserve jamais
payment_method_type	donnee carte sensible
status, amount, currency	secret fournisseur en dur
idempotency_key, timestamps	payload client non valide comme preuve finale

12. Validation backend avant confirmation frontend

```
Frontend -> "paiement termine"
X ne fait pas foi

Backend Payment
-> verifier Hyperswitch/PSP
-> reference
-> montant
-> devise
-> signature / source de confiance
-> idempotence
-> persistance PostgreSQL
-> Outbox
-> Kafka payment.captured.v1

Seulement ensuite:
Frontend -> affiche "Paieement confirme"
```

Cette regle s'applique a Payment, Order, Shipping, Returns et tout etat critique.

13. Webhooks PSP securises

Tous les webhooks passent par Kong puis le backend dedie. Avant tout traitement : authentication/signature, anti-replay, idempotence, schema validation, rate limiting et audit.

```
PSP/Hyperswitch
-> Kong
-> Payment webhook endpoint
-> verify signature/auth
-> verify timestamp/anti-replay
-> validate schema
-> deduplicate event
-> transaction PostgreSQL
-> Outbox
-> Kafka
```

Un webhook invalide ne doit modifier aucun etat metier.

14. Reconciliation periodique PSP

Ne jamais dependre uniquement des webhooks. Implementer un worker de reconciliation qui compare periodiquement l'etat interne et l'etat Hyperswitch/PSP pour les transactions non terminales ou suspectes.

```
Payment DB state
<->
Hyperswitch / PSP authoritative state
↓
MATCH -> no-op
MISMATCH -> SAFE correction / APPROVAL / MANUAL incident
```

Les corrections automatiques ne sont autorisees que lorsqu'elles sont deterministes, auditables et idempotentes.

15. Saga et compensations obligatoires

Echec	Compensation attendue
Pricing	aucun Order confirme, aucun paiement
Tax	aucun paiement, checkout fail-safe

Echec	Compensation attendue
Inventory	pas de paiement
Fraud DENY	release stock, aucun paiement
Payment FAIL	release stock, Order FAILED
Timeout apres paiement incertain	ne pas repayer; reconciliation PSP avant decision
Confirmation Order echoue apres payment	etat incident/reconciliation; jamais double capture

Le cas le plus dangereux est le timeout ambigu apres un appel paiement. Le systeme doit preferer une reconciliation de l'etat externe a un retry aveugle.

16. Transactions PostgreSQL et Outbox

```
BEGIN
write domain state
write outbox event
COMMIT

ensuite seulement:
-> publication Kafka
-> appels reseau suivants
```

Les evenements durables doivent etre publies depuis l'Outbox. Ne jamais faire "write DB puis publish Kafka" sans mecanisme de fiabilite.

17. Convention evenementielle Kafka

```
{
  "event_id": "uuidv7",
  "event_type": "order.confirmed.v1",
  "aggregate_id": "order-id",
  "aggregate_version": 7,
  "occurred_at": "UTC RFC3339",
  "producer": "order-service",
  "correlation_id": "...",
  "causation_id": "...",
  "trace_id": "..."
}
```

Partition key = aggregate_id. Les consumers sont idempotents. Les schemas Protobuf sont controles par Apicurio avec compatibilite adaptee aux evenements critiques.

18. Evenements minimaux

Service	Evenements
Order	order.created.v1, order.confirmed.v1, order.failed.v1, order.cancelled.v1
Inventory	inventory.reserved.v1, inventory.released.v1
Fraud/Risk	risk.assessed.v1, risk.denied.v1
Payment	payment.authorized.v1, payment.captured.v1, payment.failed.v1
Tax	tax.calculated.v1 si besoin durable/audit

19. Resilience et budget de timeout

Definir un budget de temps de bout en bout. Ne pas laisser chaque service choisir un timeout arbitraire. Les appels independants doivent etre paralleles si cela respecte les dependances metier.

```
Timeout obligatoire
Retry borne + backoff + jitter
Retry uniquement idempotent
Circuit breaker
Bulkhead / limite concurrence
Deadline gRPC propagee
Cancellation propagee
Un seul niveau responsable des retries
```

Interdire les retry storms : pas de 3 retries dans Order, eux-memes appeles par 3 retries dans le BFF.

20. Erreurs standardisees

```
INVALID_ARGUMENT
UNAUTHENTICATED
PERMISSION_DENIED
NOT_FOUND
CONFLICT
RATE_LIMITED
UNAVAILABLE
INTERNAL

Codes metier:
INVENTORY_NOT_AVAILABLE
RISK_DENIED
PAYMENT_ALREADY_CAPTURED
PAYMENT_PROVIDER_UNAVAILABLE
TAX_CALCULATION_FAILED
ORDER_ALREADY_CONFIRMED
```

Le frontend reagit aux codes stables, jamais au texte humain de l'erreur.

21. Securite

- Keycloak reste l'autorite d'identite/session.
- Kong valide OIDC/JWT en entree ; Istio Ambient assure mTLS/autorisation service-to-service.
- Toute autorisation critique est revalidee cote backend.
- OpenBao fournit les credentials runtime et secrets Hyperswitch/PSP avec TTL/rotation quand possible.
- Aucun PAN complet, CVV, JWT complet, secret ou credential fournisseur dans les logs.
- Redaction/masking via OpenTelemetry Collector avant Loki/Splunk.
- Egress Internet via Istio Egress Gateway puis Squid, allowlist par service.

22. Observabilite

Chaque checkout doit pouvoir etre suivi de bout en bout par correlation_id et trace_id.

```
Browser/BFF
-> Order
-> Pricing
-> Tax
-> Inventory
-> Fraud/Risk
-> Payment
-> Hyperswitch
-> PSP
```

Type	Mesures minimales
Metrics	checkout success rate, order confirmation latency, payment auth success, saga failures, compensation count
Tracing	spans par etape Saga + propagation trace_id
Logs	JSON structure, correlation_id, order_id, payment_id sans PII inutile

Type	Mesures minimales
Alerting	SLO/burn-rate, payment failures, saga stuck, reconciliation mismatch

23. Tests obligatoires

Ne pas considerer le lot termine sans automatiser les scenarios suivants.

Scenario	Attendu
Happy path	Order CONFIRMED, un seul paiement, stock reserve/commit, events publies
Double checkout	une seule commande et un seul paiement
Stock insuffisant	aucun paiement
Fraud DENY	release stock, aucun paiement
Payment FAIL	release stock, Order FAILED
PSP timeout ambigu	pas de double retry; reconciliation avant nouvel etat
Webhook duplique	ACK idempotent, pas de retraitement
Webhook signature invalide	rejet + audit, aucun changement metier
Kafka indisponible apres commit	Outbox conserve puis republication ulterieure
Restart orchestrateur Saga	workflow reprend sans duplication
Rollback release	N-1 fonctionne avec schema N

Types de tests

- Unit tests du domaine et machines d'etat.
- Integration tests avec testcontainers-go pour PostgreSQL/Kafka/RabbitMQ si utile.
- Contract tests REST/gRPC/Kafka.
- Race detector Go.
- E2E checkout avec faux PSP/Hyperswitch sandbox.
- k6 smoke sur checkout.
- Chaos ciblé: latence PSP, perte Kafka, restart orchestrateur, failover PostgreSQL.

24. CI/CD et supply chain

```
PR
-> gofmt / go vet / golangci-lint
-> unit + integration + contract tests
-> race detector
-> build
-> Trivy
-> Syft SBOM
-> Cosign
-> Harbor
-> GitOps PR / FluxCD
-> Flagger canary
-> Prometheus analysis
-> promote or rollback
```

Les images de production sont referencees par digest SHA256. Aucun latest. Kyverno doit bloquer une image non signee ou provenant d'un registry non autorise.

25. Migrations et compatibilite

```
EXPAND
-> ajouter schema compatible
MIGRATE
-> backfill idempotent/reprenable
CONTRACT
-> suppression ulterieure uniquement apres validation
```

Compatibilite N/N-1 obligatoire pendant canary et rollback. Aucun changement destructif silencieux sur REST, gRPC, Kafka ou PostgreSQL.

26. Ordre d'implementation impose

Milestone	Contenu
M1	Contrats, erreurs, modele Order, machine d'etat, migrations initiales
M2	Pricing/Tax integration + snapshots
M3	Inventory reservation + compensation
M4	Fraud/Risk decision flow
M5	Payment domain + Hyperswitch adapter + idempotence
M6	Saga orchestrator persistant + recovery
M7	Outbox/Kafka events + consumers idempotents
M8	Webhooks securises + backend validation
M9	PSP reconciliation worker
M10	Observabilite + dashboards + alertes
M11	E2E/k6/chaos ciblé
M12	Tekton/Harbor/FluxCD/Flagger + documentation

27. Definition of Done

Le lot est termine uniquement si le scenario nominal suivant fonctionne de bout en bout :

```
Cart valide
-> Checkout
-> Pricing recalcule
-> Tax recalcule
-> Inventory reserve
-> Fraud ALLOW
-> Payment autorise/capture via Hyperswitch
-> backend verifie l'etat authoritative
-> Order CONFIRMED
-> Outbox
-> Kafka order.confirmed.v1
-> frontend peut afficher CONFIRME
```

Et si les scenarios d'echec critiques sont verifies :

- Payment FAIL -> stock libere -> Order FAILED.
- Fraud DENY -> aucun paiement -> stock libere.
- Inventory FAIL -> aucun paiement.
- Double checkout -> une seule commande + un seul paiement.
- Timeout PSP ambigu -> aucune double capture; reconciliation determine l'etat reel.
- Webhook falsifie -> aucun changement metier.
- Crash de l'orchestrateur -> reprise du workflow sans duplication.

La preuve de fin de lot doit inclure les commandes exactes pour reproduire les tests et une trace OpenTelemetry d'un checkout complet.

28. Contraintes de qualite pour l'IA

- Code court, lisible, factorise et maintenable.
- Pas de copier/coller de logique metier entre handlers/transports.
- Fonctions petites et explicites; interfaces uniquement lorsqu'elles apportent une vraie decouplage.
- Pas de framework maison inutile.
- Corriger les erreurs de compilation/tests avant de poursuivre.
- Documenter les choix non triviaux dans un ADR court.
- Si une exigence est impossible ou contradictoire, la signaler avant de contourner silencieusement.
- Ne jamais simuler un test PASS : executer les tests reels dans l'environnement disponible.

FIN DU PROMPT

Commencer par M1. Ne pas coder tous les services en une seule generation. A chaque milestone: implementer -> formater -> lint -> tester -> corriger -> commit logique -> passer au milestone suivant.